

University of Maryland, Baltimore County
College of Engineering and Information Technology
Department of Computer Science and Electrical Engineering

CMPE 310: Systems Design and Programming

Lab #7



UMBC

Daniel Amissah

1. Introduction

The 8255 Programmable Peripheral Interface (PPI) is a versatile chip utilized to connect microprocessors to external I/O devices. By writing a specific control word to the 8255's command register, its three 8-bit ports (A, B, and C) can be configured for various input or output modes. This lab demonstrates how to initialize the 8255 and continuously multiplex an 8-digit 7-segment display to create scrolling text.

2. Objective

To write an 8086 GNU Assembly (GAS) code snippet that configures the 8255 PPI and scrolls the word "SHE" back and forth across an 8-digit 7-segment display. The circuit uses an I/O base address of 0x60, with Port A acting as the digit-select and Port B acting as the segment-data output.

3. Control Word Setup

The control word used in this code is 0b10000000 (or 0x80). This configuration breaks down like this:

- **Bit 7 (1):** Mode Set Flag (indicates active I/O mode, not BSR mode).
- **Bits 6-5 (00):** Group A set to Mode 0 (Basic I/O).
- **Bit 4 (0):** Port A set to Output.
- **Bit 3 (0):** Port C (Upper) set to Output.
- **Bit 2 (0):** Group B set to Mode 0 (Basic I/O).
- **Bit 1 (0):** Port B set to Output.
- **Bit 0 (0):** Port C (Lower) set to Output.

This configures all ports as basic outputs. The command byte is sent to the control register located at 0x643.

4. 7-Segment Bit Patterns for "SHE" on Port B

The standard mapping for the 8-bit data byte sent to Port B is [X g f e d c b a]. To form the letters, specific bits are set high (1) to light up the corresponding segments:

- Letter S: Segments a, b, c, d, f, g
 - Binary: 0110 1111
 - Hexadecimal: 0x6F
- Letter H: Segments b, c, e, f, g
 - Binary: 0111 0110
 - Hexadecimal: 0x76
- Letter E: Segments a, d, e, f, g
 - Binary: 0111 1001
 - Hexadecimal: 0x79

These bytes are written into an 8-byte array (MEM) and then sent sequentially out to Port B (0x641) during the display multiplexing loop.

5. Display Activation Through Port A

Port A (0x640) is used to select which of the 8 digits is currently active. The display uses active-low digit selection. The code initializes register %ah with 0xFE (binary 11111110), which selects the first digit (bit 0 is low). For each subsequent digit in the display loop, the code uses the `rol $1, %ah` (Rotate Left) instruction. This continuously "walks" the zero across the bits (11111101, 11111011, etc.), turning on one digit at a time so quickly that human persistence of vision makes them appear to be on simultaneously.

6. Movement / Shifting of "SHE"

To scroll the word "SHE", the code utilizes an outer loop (`scroll_loop`) and a scroll position counter stored in %bx, which increments from 0 to 15. During each iteration, the `UPDATE_MEM` subroutine clears the 8-byte memory buffer (MEM). It then writes the bytes for 'E', 'H', and 'S' into the buffer using %bx as an index offset. As %bx increases, the index for the letters increases, logically shifting the data patterns across the buffer. When the buffer is pushed to the display, the text appears to move across the screens.

7. Code Description

.data

MEM: .byte 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00

.text

.globl _start # GAS needs this as the entry point

_start:

 mov \$0b10000000, %al

 mov \$0x643, %dx # 0x703 -> 0x643, lab uses base address 0x640

 out %al, %dx

 mov \$0, %bx # scroll position counter, 0-15

scroll_loop: # outer loop to keep SHE bouncing

 call UPDATE_MEM

 call DISP

 inc %bx

 cmp \$16, %bx

 jl scroll_loop

 mov \$0, %bx

 jmp scroll_loop

UPDATE_MEM: # writes SHE into MEM at current scroll position

 push %ax

 push %bx

 push %si

 lea MEM, %si

 mov \$0x00, %ax

 mov %al, 0(%si) # blank all 8 digits before placing letters

 mov %al, 1(%si)

 mov %al, 2(%si)

 mov %al, 3(%si)

 mov %al, 4(%si)

```

mov %al, 5(%si)
mov %al, 6(%si)
mov %al, 7(%si)
cmp $0, %bx
jl done_update
cmp $7, %bx
jg done_update
lea MEM, %si
mov $0x79, %al      # E: segments a,d,e,f,g
mov %al, (%si,%bx)
dec %bx
cmp $0, %bx
jl skip_h
mov $0x76, %al      # H: segments b,c,e,f,g
mov %al, (%si,%bx)
skip_h:
dec %bx
cmp $0, %bx
jl skip_s
mov $0x6F, %al      # S: segments a,b,c,d,f,g
mov %al, (%si,%bx)
skip_s:
add $2, %bx
done_update:
pop %si
pop %bx
pop %ax
ret

```

DISP:

```

pushf
push %ax
push %bx
push %dx
push %si

```

```

    mov $8,    %bx
    mov $0xFE, %ah      # 0x7F -> 0xFE, Port A bit0 is dig-0 so active-low
starts here
    lea MEM,   %si
    mov $0x640, %dx      # 0x701 -> 0x640, Port A is digit-select in Fig A

```

```

disp_loop:
    mov %ah, %al
    out %al, %dx          # digit-select to Port A (was Port B in textbook)

    inc %dx               # DEC -> INC, Port B is one above Port A at 0x641
    mov (%si), %al
    out %al, %dx          # segment data to Port B (was Port A in textbook)

    call DELAY

    rol $1, %ah           # ROR -> ROL, walking active-low bit from bit0 to bit7

    dec %dx               # INC -> DEC, back to Port A for next digit
    inc %si

    dec %bx
    jnz disp_loop

    pop %si
    pop %dx
    pop %bx
    pop %ax
    popf
    ret

```

```

DELAY:                # defined here since original only called but never wrote it
    push %cx
    mov $0xFFFF, %cx

```

```
delay_loop:
    dec %cx
    jnz delay_loop
    pop %cx
    ret
```

8. Questions

1. What segments must be lit for S, H, and E?

- **a. S:** a, b, c, d, f, g
- **b. H:** b, c, e, f, g
- **c. E:** a, d, e, f, g

2. What byte must be sent to Port B for each letter?

- **a. S:** g=1, f=1, e=0, d=1, c=1, b=1, a=1; **0x6F**
- **b. H:** g=1, f=1, e=1, d=0, c=1, b=1, a=0; **0x76**
- **c. E:** g=1, f=1, e=1, d=1, c=0, b=0, a=1; **0x79**

3. What mode do Port A and Port B need to be in? Why?

- Port A and Port B need to be in Mode 0 because we don't need any handshaking or strobing; we are simply writing the data straight to the output pins.

4. Can you reprogram the 8255 to support a 9-digit display?

- Not with Port A since it only has 8 bits; however, with Port C you can reprogram it to support a 9-digit display by using its pins as an output.

9. Conclusion

This lab successfully demonstrates the application of the 8255 Programmable Peripheral Interface for managing visual I/O. By configuring the chip in Mode 0, direct output writing can be utilized to independently control both the physical

segments (Port B) and the active digits (Port A). By synchronizing a small delay loop with the cycling of the active digit and iterating through memory offsets, we successfully implemented a multiplexing and scrolling animation logic via GNU Assembly.